西安交通大学实验报告

课程 Rust 程序设计 实验名称__实验一____ 第 1 页 共 28 页

系 别_____计算机科学与技术_____ 实 验 日 期 2026 年 3 月 25 日
专业班级_____组别_____6_____ 实 验 报 告 日 期 2026 年 3 月 25 日
姓 名_____学号 报 告 退 发（订正 、 重做 ）
同 组 人____、、（该实验独立完成）_____
教 师 审 批 签 字

实验名称：实验一

一. 题目

任务一：请在本地完成 Rust 编译环境的安装，并进行测试。

任务二：安装 VSCode，并安装 Rust-analyzer 插件，安装完成后，进行环境测试。

任务三：编写一个杨辉三角生成器，输入层数 n，输出对应层数的杨 辉三角。

任务四：编写程序，输入一个字符串，将字符串中的小写字母转换为大写字母，大写字母转换为小写字母，其他字符不变。

任务五：编写程序，为小学生随机出加、减、乘、除法的题目，即每次出的题目不仅两个数是随机的，计算方法也是随机的，即可能是加，也可能是减，或乘，或除。数在 20 以内，减法要保证被减数大，除法要保证被除数大且能整除

任务六：编写程序将百分制转换成五分制。

成绩 90~100 分，输出 5 分；

成绩 80~ 89 分，输出 4 分；

成绩 70~ 79 分，输出 3 分；

成绩 60~ 69 分，输出 2 分；

成绩 0 ~ 59 分，输出 1 分；

输入一个百分制成绩，输出一个五分制成绩。

任务七：编写程序，从键盘输入 n，计算级数的前 n 项的和并输出。

$$S = 1 + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} + \cdots + \frac{1}{n^2}$$

任务八：编写程序，实现“石头剪刀布”游戏，游戏采用三局两胜制，请实现上述功能，列出每一局的输赢情况及最后的输赢情况。

二. 环境

- 操作系统：Ubuntu 24.04.3 LTS
- 运行平台：WSL2
- 内核版本：Linux 6.6.87.2-microsoft-standard-WSL2
- 处理器：13th Gen Intel(R) Core(TM) i5-13500HX
- CPU 核心：4 逻辑 CPU
- 内存：7.8 GiB
- 交换空间：4.0 GiB
- Rust 工具链：stable-x86_64-unknown-linux-gnu
- 编译器版本：rustc 1.94.0
- 构建工具版本：cargo 1.94.0

三. 实验步骤

任务一：本实验首先在本机完成 Rust 编译环境安装，安装 Rust 稳定版工具链。安装完成后，使用命令对环境进行检查。

任务二：在完成 Rust 编译环境安装后，安装 Visual Studio Code 作为开发工具，并安装 rust-analyzer 插件用于 Rust 代码分析与编辑支持。

任务三：杨辉三角代码：

```
use std::io::{self, Write};



fn generate_pascal_triangle(n: usize) -> Vec<Vec<u64>> {
    let mut triangle: Vec<Vec<u64>> = Vec::new();
    for i in 0..n {
        let mut row: Vec<u64> = Vec::new();
        if i == 0 {row.push(1);}
        else {
            let prev = &triangle[i - 1];
            row.push(1);
            for w in prev.windows(2) {
                row.push(w[0] + w[1]);
            }
            row.push(1);
        }
        triangle.push(row);
    }
```

```
    triangle
}

fn main() {
    print!("请输入杨辉三角的行数: ");
    io::stdout().flush().expect("刷新失败");

    let mut input = String::new();
    io::stdin().read_line(&mut input).expect("读取失败");

    let n: usize = input.trim().parse().expect("请输入数字");

    let triangle = generate_pascal_triangle(n);
    println!("杨辉三角的前 {} 行:", n);

    for row in &triangle {
        let mut parts: Vec<String> = Vec::new();
        for num in row {
            parts.push(num.to_string());
        }
        println!("{}", parts.join(" "));
    }
```

```
}
```

代码说明：

使用 Vec<Vec<u64>> 作为二维动态数组存储杨辉三角，核心逻辑在 generate_pascal_triangle 函数中：首行直接压入 1，其余各行利用 Rust 标准库的 windows(2) 滑动窗口迭代器遍历上一行相邻两个元素并求和，生成中间元素，首尾各补 1。

main 函数通过 io::stdout().flush() 手动刷新标准输出缓冲区以确保提示文字在读取输入前显示，再将输入字符串经 trim().parse() 转换为 usize 类型。输出时以 &triangle 借用方式遍历，避免所有权转移，用 join(" ") 将每行数字以空格拼接后输出。

任务四：大小写转换代码：

```
use std::io::{self, Write};

fn read_input(prompt: &str) -> String {
    print!("{}", prompt);
    io::stdout().flush().expect("刷新失败");

    let mut input = String::new();
    io::stdin().read_line(&mut input).expect("读取失败");
    input.trim().to_string()
}
```

```rust
fn swap_case(s: &str) -> String {
    let mut result = String::new();

    for c in s.chars() {
        if c.is_lowercase() {
            result.push(c.to_uppercase().next().unwrap());
        } else if c.is_uppercase() {
            result.push(c.to_lowercase().next().unwrap());
        } else {
            result.push(c);
        }
    }

    result
}

fn main() {
    let input = read_input("请输入字符串: ");
    let output = swap_case(&input);
    println!("转换结果: {}", output);
}
```

代码说明：

将读取提示并刷新缓冲区的逻辑封装为 read_input 函数，提升复用性，后续代码中涉及输出提示信息部分都运用该函数实现。

核心转换函数 swap_case 接收字符串切片&str，通过.chars()迭代器逐一处理每个字符：调用 is_lowercase() 和 is_uppercase() 判断大小写，再通过 to_uppercase()和 to_lowercase()方法转换，由于这两个方法返回迭代器而非直接返回字符，使用 .next().unwrap() 取出首个字符，最后 push 到结果字符串中。非字母字符直接原样保留。

任务五：小学数学题代码：

```
use rand::Rng;
use std::io::{self, Write};


fn read_input(prompt: &str) -> String {
    print!("{}", prompt);
    io::stdout().flush().expect("刷新失败");
    let mut input = String::new();
    io::stdin().read_line(&mut input).expect("读取失败");
    input.trim().to_string()
}


fn generate_addition<R: Rng + ?Sized>(rng: &mut R) -> (i32, i32, &'static str, f64) {
    let a = rng.gen_range(1..=20);
```

```
    let b = rng.gen_range(1..=20);
    (a, b, "+", (a + b) as f64)
}

fn generate_subtraction<R: Rng + ?Sized>(rng: &mut R) -> (i32, i32, &'static str, f64) {
    let a = rng.gen_range(2..=20);
    let b = rng.gen_range(1..a);
    (a, b, "-", (a - b) as f64)
}

fn generate_multiplication<R: Rng + ?Sized>(rng: &mut R) -> (i32, i32, &'static str, f64) {
    let a = rng.gen_range(1..=20);
    let b = rng.gen_range(1..=20);
    (a, b, "×", (a * b) as f64)
}

fn generate_division<R: Rng + ?Sized>(rng: &mut R) -> (i32, i32, &'static str, f64) {
    loop {
        let b = rng.gen_range(1..=10);
        let max_multiplier = 20 / b;

        if max_multiplier >= 2 {
```

```
            let multiplier = rng.gen_range(2..=max_multiplier);
            let a = b * multiplier;
            return (a, b, "÷", (a / b) as f64);
        }
    }
}

fn generate_question() -> (String, f64) {
    let mut rng = rand::thread_rng();
    let op = rng.gen_range(0..4);  // 0=加 1=减 2=乘 3=除

    let (a, b, symbol, answer) = match op {
        0 => generate_addition(&mut rng),
        1 => generate_subtraction(&mut rng),
        2 => generate_multiplication(&mut rng),
        _ => generate_division(&mut rng),
    };

    let question = format!("{} {} {} = ?", a, symbol, b);
    (question, answer)
}
```

```rust
fn main() {
    println!("=== 小学数学练习 ===");
    println!("输入 q 退出\n");

    let mut correct = 0;
    let mut total = 0;

    loop {
        let (question, answer) = generate_question();
        let input = read_input(&format!("{} ", question));
        if input == "q" {
            break;
        }

        match input.parse::<f64>() {
            Ok(user_answer) => {
                total += 1;
                if user_answer == answer {
                    println!("✓ 正确！\n");
                    correct += 1;
                } else {
                    println!("✗ 错误，正确答案是 {}\n", answer);
```

```
            }
        }
        Err(_) => {
            println!("请输入数字\n");
        }
    }
}

if total > 0 {
    println!("答题结束！答对 {}/{} 题", correct, total);
}
}
```

代码说明：

引入第三方库 rand 生成随机数，需在 Cargo.toml 中添加依赖 rand = "0.8"。

将加、减、乘、除四种运算分别封装为独立函数，其中加法和乘法直接在给定范围内随机生成两个操作数；减法通过限制被减数 a 取值为 2..=20、减数 b 取值为 1..a，保证结果始终为正且两个数均不超过 20；除法通过先随机生成除数 b，再计算满足 a<=20 的最大倍数 max_multiplier，随后令被除数 a=b*multiplier，从而同时保证题目中的两个数均在 20 以内，且除法能够整除。

generate_question 函数利用 rand::thread_rng().gen_range(0..4) 随机选择四种运算，并通过 match 调用对应的生成函数，最终以元组形式返回题目字符串和

标准答案。

main 函数使用 loop 循环持续出题，通过 match 对 parse::<f64>() 的结果进行模式匹配，区分合法答案与非法输入；当用户输入 q 时结束练习，并输出最终答题统计结果。

任务六：百分制转换五分制代码：

```
use std::io::{self, Write};



fn read_input(prompt: &str) -> String {

    print!("{}", prompt);

    io::stdout().flush().expect("刷新失败");

    let mut input = String::new();

    io::stdin().read_line(&mut input).expect("读取失败");

    input.trim().to_string()

}



fn to_five_scale(score: u32) -> Option<u32> {

    match score {

        90..=100 => Some(5),

        80..=89 => Some(4),

        70..=79 => Some(3),

        60..=69 => Some(2),
```

```
        0..=59 => Some(1),
        _ => None,
    }
}


fn main() {
    let input = read_input("请输入成绩 (0-100): ");
    match input.parse::<u32>() {
        Ok(score) => {
            match to_five_scale(score) {
                Some(scale) => println!("五分制成绩: {}", scale),
                None => println!("输入的成绩不在有效范围内"),
            }
        }
        Err(_) => println!("输入的不是有效的数字"),
    }
}
```

代码说明：

定义 to_five_scale 函数，返回类型为 Option<u32>，利用 match 表达式的闭区间范围匹配语法将百分制成绩映射到五分制：90..=100 对应 5 分，依次递减，非法输入返回 None。

main 函数对 parse::<u32>() 的结果做第一层 match 处理解析错误，再对

to_five_scale 的 Option 结果做第二层 match 处理越界情况，实现完整的错误处理链，避免程序异常崩溃。

任务七：级数前 n 项和代码：

```
use std::io::{self, Write};

fn read_input(prompt: &str) -> String {

    print!("{}", prompt);

    io::stdout().flush().expect("刷新失败");

    let mut input = String::new();

    io::stdin().read_line(&mut input).expect("读取失败");

    input.trim().to_string()

}



fn sum_series(n: u32) -> f64 {

    (1..=n).map(|i| 1.0 / (i as f64).powi(2)).sum()

}



fn main() {

    let input = read_input("请输入正整数 n: ");

    match input.parse::<u32>() {

        Ok(n) if n > 0 => {

            let result = sum_series(n);
```

```
            println!("前 {} 项的倒数平方和为: {:.6}", n, result);
        }
        _ => println!("请输入一个有效的正整数"),
    }
}
```

代码说明：

定义 sum_series 函数，利用 Rust 迭代器链一行实现级数求和：(1..=n) 生成从 1 到 n 的整数范围，.map(|i| 1.0 / (i as f64).powi(2))将每个整数转为 f64 类型后求其平方的倒数，.sum()对所有元素求和并返回 f64 结果。

任务八：石头剪刀布游戏代码：

```
use rand::Rng;
use std::io::{self, Write};


#[derive(Debug, Clone, PartialEq)]
enum Move {
    Rock,     // 石头
    Scissors, // 剪刀
    Paper,    // 布
}


#[derive(Debug, PartialEq)]
```

```rust
enum RoundResult {
    Win,
    Lose,
    Draw,
}


fn read_input(prompt: &str) -> String {
    print!("{}", prompt);
    io::stdout().flush().expect("刷新失败");
    let mut input = String::new();
    io::stdin().read_line(&mut input).expect("读取失败");
    input.trim().to_lowercase()
}


fn parse_move(input: &str) -> Option<Move> {
    match input {
        "1" | "rock"     | "石头" => Some(Move::Rock),
        "2" | "scissors" | "剪刀" => Some(Move::Scissors),
        "3" | "paper"    | "布"   => Some(Move::Paper),
        _ => None,
    }
}
```

```rust
fn computer_move() -> Move {
    match rand::thread_rng().gen_range(0..3) {
        0 => Move::Rock,
        1 => Move::Scissors,
        _ => Move::Paper,
    }
}

fn judge(player: &Move, computer: &Move) -> RoundResult {
    match (player, computer) {
        // 平局
        (p, c) if p == c => RoundResult::Draw,
        // 赢
        (Move::Rock,     Move::Scissors) => RoundResult::Win,
        (Move::Scissors, Move::Paper)    => RoundResult::Win,
        (Move::Paper,    Move::Rock)     => RoundResult::Win,
        // 输
        _ => RoundResult::Lose,
    }
}
```

```rust
fn move_name(m: &Move) -> &str {
    match m {
        Move::Rock     => "石头",
        Move::Scissors => "剪刀",
        Move::Paper    => "布",
    }
}


fn main() {
    println!("石头剪刀布（三局两胜）");
    println!("输入: 1=石头  2=剪刀  3=布\n");

    let mut player_wins = 0;
    let mut computer_wins = 0;
    let mut round = 1;

    // 三局两胜：任意一方赢 2 局就结束
    while player_wins < 2 && computer_wins < 2 {
        println!("--- 第 {} 局 ---", round);

        // 读取玩家输入，输入非法则重新输入
        let player = loop {
```

```
        let input = read_input("请出拳: ");
        match parse_move(&input) {
            Some(m) => break m,          // 合法输入
            None    => println!("无效输入，请输入 1、2 或 3"),
        }
    };

    let computer = computer_move();

    println!("你出了: {}  电脑出了: {}", move_name(&player), move_name(&computer));

    match judge(&player, &computer) {
        RoundResult::Win  => {
            println!("结果: 你赢了这局！\n");
            player_wins += 1;
        }
        RoundResult::Lose => {
            println!("结果: 你输了这局！\n");
            computer_wins += 1;
        }
        RoundResult::Draw => {
            println!("结果: 平局，本局不计\n");
```

```
            // 平局不增加任何计数，也不推进 round
            continue;
        }
    }

    round += 1;
}

println!("最终结果");
println!("你 {} : {} 电脑", player_wins, computer_wins);
if player_wins > computer_wins {
    println!("恭喜你赢得了比赛！");
} else {
    println!("电脑赢得了比赛！");
}
}
```

代码说明：

使用两个自定义枚举 Move（石头/剪刀/布）和 RoundResult（Win/Lose/Draw）表达游戏状态，通过 #[derive(Debug, Clone, PartialEq)] 属性宏自动派生调试输出、克隆和相等比较能力。

judge 函数对 (player, computer) 元组进行模式匹配，使用守卫条件 if p == c 处理平局，枚举所有获胜组合，其余情况统一视为失败。

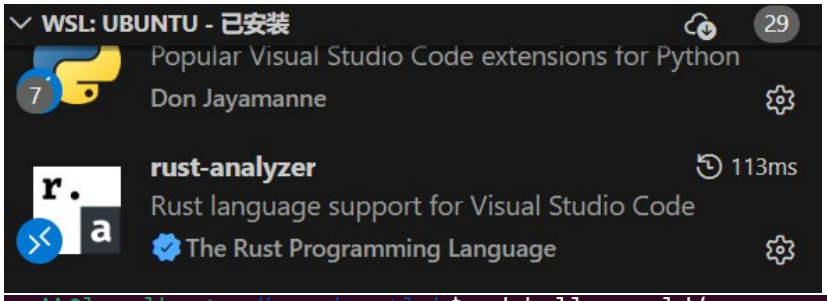
main 函数以 while player_wins < 2 && computer_wins < 2 实现三局两胜循环，内层用 loop + break 返回值的写法持续接收输入直到合法，平局通过 continue 跳过计数使该局重新进行。

四. 数据、结果分析

任务一：环境配置信息

```
wuji@localhost:~/home/rustlab/lab1$ rustc --version
rustc 1.94.0 (4a4ef493e 2026-03-02)
wuji@localhost:~/home/rustlab/lab1$ cargo --version
cargo 1.94.0 (85eff7c80 2026-01-15)
wuji@localhost:~/home/rustlab/lab1$ rustup show active-toolchain
stable-x86_64-unknown-linux-gnu (default)
```

任务二：插件安装与编译测试




```
wuji@localhost:~/home/rustlab$ cd hello_world/
wuji@localhost:~/home/rustlab/hello_world$ ls
Cargo.lock  Cargo.toml  src  target
wuji@localhost:~/home/rustlab/hello_world$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.03s
     Running `target/debug/hello_world`
Hello, world!
wuji@localhost:~/home/rustlab/hello_world$ cat src/main.rs
fn main() {
    println!("Hello, world!");
}
```

任务三：杨辉三角运行结果

```
*  正在执行任务: cargo run --package task1_pascal_triangle --bin task1_pascal_triangle

   Compiling task1_pascal_triangle v0.1.0 (/home/wuji/home/rustlab/lab1/task1_pascal_triangle)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.10s
     Running `target/debug/task1_pascal_triangle`
请输入杨辉三角的行数: 6
杨辉三角的前 6 行:
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

结果分析：程序正确生成并输出了指定行数的杨辉三角。每行首尾均为 1，中间元素等于上一行相邻两元素之和，符合杨辉三角的数学定义。输出格式以空格分隔各元素，每行独立输出，结果直观清晰。

任务四：大小写转换运行结果

```
*  正在执行任务: cargo run --package task2_swap_case --bin task2_swap_case

   Compiling task2_swap_case v0.1.0 (/home/wuji/home/rustlab/lab1/task2_swap_case)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.17s
     Running `target/debug/task2_swap_case`
请输入字符串: ABcEDergsfDFSwsDwhN;1L'-
转换结果: abCedERGSFdfsWSdWHn;1l'-
```

结果分析：程序正确实现了大小写互换功能。输入字符串中的小写字母均被转换为对应大写字母，大写字母均被转换为对应小写字母，数字、标点等非字母字符保持不变，符合题目要求。

任务五：小学算术题运行结果

```
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.04s
     Running `target/debug/task3_random_arithmetic`
=== 小学数学练习 ===
输入 q 退出
 …
✓ 正确！

18 ÷ 6 = ? 3
✓ 正确！

10 ÷ 2 = ? 2
✗ 错误，正确答案是 5

18 ÷ 9 = ? 2
✓ 正确！

11 × 7 = ? 77
✓ 正确！

2 + 16 = ? 18
✓ 正确！

3 + 8 = ? 11
✓ 正确！

14 ÷ 7 = ? 2
✓ 正确！

5 + 14 = ? 19
✓ 正确！

15 + 14 = ? q
答题结束！答对 14/15 题
```

结果分析：程序能够随机生成加、减、乘、除四种运算题目，两个操作数均在题目要求范围内随机产生。减法结果均为非负整数，除法结果均为整数，满足题目约束条件。

任务六：百分制转换五分制运行结果

```
wuji@localhost:~/home/rustlab/lab1/task4_score_to_five_scale$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task4_score_to_five_scale`
请输入成绩 (0-100): 95
五分制成绩: 5
wuji@localhost:~/home/rustlab/lab1/task4_score_to_five_scale$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task4_score_to_five_scale`
请输入成绩 (0-100): 80
五分制成绩: 4
wuji@localhost:~/home/rustlab/lab1/task4_score_to_five_scale$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task4_score_to_five_scale`
请输入成绩 (0-100): 79
五分制成绩: 3
wuji@localhost:~/home/rustlab/lab1/task4_score_to_five_scale$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task4_score_to_five_scale`
请输入成绩 (0-100): 62
五分制成绩: 2
wuji@localhost:~/home/rustlab/lab1/task4_score_to_five_scale$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task4_score_to_five_scale`
请输入成绩 (0-100): 0
五分制成绩: 1
```

结果分析：程序对 0~100 范围内的各区间成绩均能正确转换为对应五分制成绩，边界值的判断结果准确。

任务七：级数和运行结果

```
wuji@localhost:~/home/rustlab/lab1/task5_inverse_square_sum$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task5_inverse_square_sum`
请输入正整数 n: 1
前 1 项的倒数平方和为: 1.000000
wuji@localhost:~/home/rustlab/lab1/task5_inverse_square_sum$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task5_inverse_square_sum`
请输入正整数 n: 2
前 2 项的倒数平方和为: 1.250000
wuji@localhost:~/home/rustlab/lab1/task5_inverse_square_sum$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task5_inverse_square_sum`
请输入正整数 n: 3
前 3 项的倒数平方和为: 1.361111
wuji@localhost:~/home/rustlab/lab1/task5_inverse_square_sum$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task5_inverse_square_sum`
请输入正整数 n: 4
前 4 项的倒数平方和为: 1.423611
wuji@localhost:~/home/rustlab/lab1/task5_inverse_square_sum$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task5_inverse_square_sum`
请输入正整数 n: 5
前 5 项的倒数平方和为: 1.463611
wuji@localhost:~/home/rustlab/lab1/task5_inverse_square_sum$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.08s
     Running `/home/wuji/home/rustlab/lab1/target/debug/task5_inverse_square_sum`
请输入正整数 n: 100000
前 100000 项的倒数平方和为: 1.644924
```

结果分析：程序正确计算了级数前 n 项之和，输出结果保留六位小数，精

度满足要求。随着 n 增大，计算结果逐渐趋近于数学极限值 $\pi^2/6 \approx 1.644934$，验证了程序计算逻辑的正确性。

任务八：石头剪刀布运行结果

```
   Compiling task6_rock_paper_scissors v0.1.0 (/home/wuji/home/rustlab/lab1/task6_rock_paper_scissors)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.09s
     Running `target/debug/task6_rock_paper_scissors`
石头剪刀布（三局两胜）
输入：1=石头  2=剪刀  3=布

--- 第 1 局 ---
请出拳：1
你出了：石头  电脑出了：布
结果：你输了这局！

--- 第 2 局 ---
请出拳：2
你出了：剪刀  电脑出了：石头
结果：你输了这局！

最终结果
你 0 : 2 电脑
电脑赢得了比赛！
```

```
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
     Running `target/debug/task6_rock_paper_scissors`
石头剪刀布（三局两胜）
输入：1=石头  2=剪刀  3=布

--- 第 1 局 ---
请出拳：2
你出了：剪刀  电脑出了：布
结果：你赢了这局！

--- 第 2 局 ---
请出拳：2
你出了：剪刀  电脑出了：剪刀
结果：平局，本局不计

--- 第 2 局 ---
请出拳：1
你出了：石头  电脑出了：石头
结果：平局，本局不计

--- 第 2 局 ---
请出拳：1
你出了：石头  电脑出了：剪刀
结果：你赢了这局！

最终结果
你 2 : 0 电脑
恭喜你赢得了比赛！
```

结果分析：程序正确实现了三局两胜制石头剪刀布游戏。每局均输出玩家和电脑的出拳结果及本局胜负，平局时本局不计入胜负计数并重新进行。当任意一方率先赢得两局时游戏结束，并输出最终比分和胜负结论。电脑出拳由随机数驱动，结果随机性正常。

五. 总结

本次实验共完成六个编程任务，系统地完成了 Rust 基础编程能力的训练。通过本次实验，主要收获如下：

1. 掌握了 Rust 的基础语法特性。包括变量声明（let / let mut）、基本数据类型（u32、usize、f64 等）、函数定义与返回值、字符串处理（String 与 &str 的区别）、格式化输出（println! / print!）等，初步建立了 Rust 编程的基本范式。

2. 理解了 Rust 所有权与借用机制。实验中频繁使用 & 借用操作符，理解了借用不转移所有权的核心概念，避免了 for 循环中因所有权转移导致变量失效的问题，这是 Rust 区别于 C/C++ 和 Java 最重要的语言特性之一。

3. 熟悉了 Rust 的错误处理机制。通过 Option<T> 和 Result<T, E> 两种类型，结合 match 模式匹配，实现了对非法输入、解析失败等异常情况的完整处理，养成了不依赖 panic 崩溃程序的良好编码习惯。

4.学习了 Rust 迭代器的基本使用。通过 .map()、.sum()、.windows()、.chars() 等迭代器方法，以声明式风格替代传统索引循环，代码更加简洁、安全，不存在越界风险。

5. 了解了 Rust 枚举与模式匹配。在石头剪刀布任务中使用自定义枚举和 match 表达式，体会到 Rust 的 match 相比 C 的 switch 在表达能力和安全性上的显著提升，包括范围匹配、元组匹配和守卫条件等特性。

Rust 语言在保证内存安全的前提下提供了强大的表达能力，编译器严格的类型检查和借用检查虽然初期增加了学习负担，但有效避免了许多在 C/C++ 中常见的运行时错误，对系统编程能力的提升具有重要意义。